Z3 Bit-Vector Quick Reference (Python)

Start every program with `from z3 import *`. A bit-vector is a **fixed-width** string of bits; arithmetic **wraps around modulo 2^n** (n = width). The bits have no sign by themselves — the *operator* decides whether they are read as signed (two's complement) or unsigned.

1. Declaring variables and constants

Operation	Syntax	Example	Meaning
Variable	`BitVec('x', n)`	`x = BitVec('x', 8)`	unknown n-bit value (0..2^n-1 as unsigned)
Several variables	`BitVecs('x y', n)`	`x, y = BitVecs('x y', 8)`	declare many at once
Constant	`BitVecVal(v, n)`	`BitVecVal(10, 8)`	8-bit constant 00001010
Literals	decimal / `0b` / `0x`	`BitVecVal(0xFF, 8)`	11111111 (255 unsigned, -1 signed)
Python int mixed in	`x + 5`	`x == 10`	int is auto-converted to x's width

2. Arithmetic (wraps modulo 2^n)

Operation	Syntax	Example	Meaning
Add	`x + y`	`x + 1 == 0`	8-bit: sat with x = 255 (255+1 wraps to 0)
Subtract	`x - y`	`x - y`	0 - 1 = 255 (8-bit)
Multiply	`x * y`	`x * 2`	low n bits of the product
Negate	`-x`	`-x`	two's complement negation
Signed division	`x / y`	`x / 2`	signed (254 = -2, so -2 / 2 = -1)
Unsigned division	`UDiv(x, y)`	`UDiv(x, 2)`	254 / 2 = 127
Signed remainder	`SRem(x, y)`	`SRem(x, 3)`	sign follows dividend: -7 SRem 3 = -1
Signed modulo	`x % y`	`x % 3`	Python-style `%` is signed modulo, sign follows divisor: -7 % 3 = 2
Unsigned remainder	`URem(x, y)`	`URem(x, 16)`	remainder as unsigned

3. Bitwise operations

Operation	Syntax	Example	Meaning
AND	`x & y`	`x & 0x0F`	1010 & 1100 = 1000 (mask low nibble)
OR	`x | y`	`x | 0x80`	1010 | 1100 = 1110 (set bits)

XOR	`x ^ y`	`x ^ y`	1010 ^ 1100 = 0110 (differing bits)
NOT	`~x`	`~x`	flip every bit: ~1010 = 0101

4. Shifts and rotates (on 10110000)

Operation	Syntax	Example	Result
Shift left	`x << n`	`x << 2`	11000000 (top bits lost, zeros in)
Logical shift right	`LShR(x, n)`	`LShR(x, 2)`	00101100 (zeros fill left)
Arithmetic shift right	`x >> n`	`x >> 2`	11101100 (sign bit copied)
Rotate	`RotateLeft(x, n)` / `RotateRight`	`RotateLeft(x, 3)`	bits wrap around, none lost

Careful: in Z3, `>>` is the *arithmetic* shift, not logical!

5. Changing width / picking bits

Operation	Syntax	Example	Meaning
Extract bits	`Extract(hi, lo, x)`	`Extract(7, 4, x)`	bits hi..lo (inclusive, **high first**); width = hi-lo+1. Single bit: `Extract(0,0,x)`
Concatenate	`Concat(x, y)`	`Concat(x, y)`	x becomes the high bits; widths add (4+4 = 8)
Zero extend	`ZeroExt(k, x)`	`ZeroExt(8, x)`	add k zero bits on the left: 10101100 → 0000000010101100
Sign extend	`SignExt(k, x)`	`SignExt(8, x)`	copy sign bit k times: 10101100 → 1111111110101100
Repeat	`RepeatBitVec(k, x)`	`RepeatBitVec(2, x)`	concatenate x with itself k times

6. Comparisons — signed vs unsigned

Operation	Syntax	Example	Meaning
Equal / not equal	`x == y`, `x != y`	`x == 5`	builds a constraint (does not return True/False)
Signed < ≤ > ≥	`x < y`, `x <= y`, `x > y`, `x >= y`	`x < 0`	two's complement; 11111111 is -1
Unsigned <	`ULT(x, y)`	`ULT(x, 10)`	11111111 is 255
Unsigned ≤	`ULE(x, y)`	`ULE(x, y)`	
Unsigned >	`UGT(x, y)`	`UGT(x, y)`	

Unsigned ≥	`UGE(x, y)`	`UGE(x, y)`

Example: with x = 11111111, `x < 0` is **True** (-1 < 0) but `ULT(x, 0)` is **False** (255 < 0).

7. Combining constraints (Boolean logic)

Operation	Syntax	Example
And / Or / Not	`And(a, b)`, `Or(a, b)`, `Not(a)`	`And(x > 0, ULT(x, 10))`
Implication	`Implies(a, b)`	`Implies(x == 0, y == 0)`
If-then-else	`If(cond, a, b)`	`If(x < 0, -x, x)` (absolute value)

Do **not** use Python's `and`, `or`, `not` on Z3 expressions — use `And`, `Or`, `Not`.

8. Solving and reading results

```
from z3 import *

x = BitVec('x', 8)
y = BitVec('y', 8)

s = Solver()
s.add(x + y == 10, ULT(x, y), x != 0)   # add constraints

if s.check() == sat:                      # sat / unsat / unknown
    m = s.model()
    print(m)                              # e.g. [x = 1, y = 9]
    print(m[x].as_long())                 # model value as Python int
    print(m.eval(x * 2))                  # evaluate any expression
else:
    print("no solution")

print(simplify(BitVecVal(200, 8) + BitVecVal(100, 8)))   # 44 (computes constants)
```

Operation	Syntax
Add constraint	`s.add(c1, c2, ...)`
Check	`s.check()` → `sat`, `unsat`, `unknown`
Get model	`s.model()`, `m[x].as_long()`, `m.eval(expr)`
Prove a property	add its **negation**; `unsat` means the property always holds
Convert to/from Int	`BV2Int(x)` (unsigned), `Int2BV(i, n)`

9. Overflow — what to remember

- **Wrap-around:** an n-bit result is reduced mod 2^n. 8-bit: 255 + 1 = 0, 200 + 100 = 44, 0 - 1 = 255.
- **Unsigned overflow** of x + y happened iff `ULT(x + y, x)`. Z3 also has `BVAddNoOverflow(x, y, False)`.
- **Signed overflow:** 8-bit range is -128..127, so 127 + 1 = -128. Check with `BVAddNoOverflow(x, y, True)` and `BVAddNoUnderflow(x, y)`.
- **Int vs BitVec:** `Int` never overflows (`i + 1 == 0` gives i = -1); an 8-bit `BitVec` gives x = 255.

10. Common mistakes

- **Mixed widths** (e.g. 8-bit + 4-bit) raise an error — use `ZeroExt`/`SignExt`/`Extract` first.
- **Signed vs unsigned:** `< > / % >>` are signed/arithmetic; use `ULT UGT UDiv URem LShR` for unsigned/logical.
- **Extract order** is `Extract(high, low, x)`, not low first.
- `x == y` is a Z3 constraint, not a Python Boolean — don't use it in a Python `if`; use `s.check()` or `simplify`.
- The width of a constant must match: `BitVecVal(300, 8)` silently wraps to 44.